

LAB MANUAL

CSE 324

Operating Systems Lab

Course Teacher

Nushrat Jahan Oyshi

Lecturer

Computer Science & Engineering

Daffodil International University

Lab 2

Variables:

- System variable: Mostly written using upper cases, 'VAR'
- User defined variable:
 - No fixed format
 - Mostly written using lower cases, 'var'

Commands: System variable

Script: *hello.sh*

```
#!/bin/bash
echo "Hello World "

echo $BASH      # gives the name of the bash
echo $BASH_VERSION
echo $HOME      # gives the name of the home directory
echo $PWD       # gives the name of the present directory
```

User Defined Variable:

Script: *hello.sh*

```
#!/bin/bash
echo "Hello World "

n = abcd
10val = 10
echo The name is $n
echo The value is $10val

val = 10
echo The value is $val
```

Execute: *./hello.sh*

Read Command:

Script: *hello.sh*

```
#!/bin/bash
echo "Enter name: "
read n          # store the input in n variable
echo "Entered name: $n"
```

Script: *Terminal*

```
Enter name:
Input: abcd
Output:
Entered name: abcd
```

Read from Default Variable:

Script: *hello.sh*

```
#!/bin/bash
echo "Enter name: "
read          # store the input in a default variable $REPLY
echo "Name : $REPLY"
```

Script: *Terminal* *./hello.sh*

```
Input:
Enter name:
abcd
```

```
Output:
Name : abcd
```

Read Multiple Inputs:

Script: <i>hello.sh</i> <pre>#!/bin/bash echo "Enter names: " read n1 n2 n3 # store the input in n variable echo "Name: \$n1, \$n2, \$n3"</pre> Script: <i>Terminal</i> <pre>Enter name: abc def ghi Output: Names: abc , def , ghi</pre>	Script: <i>hello.sh</i> <pre>#!/bin/bash # -p flag allows to enter input on the same line as the prompt read -p 'username : ' user_var # invisible / silent user input read -sp 'password : ' pass_var echo echo "username : \$user_var" echo "password : \$pass_var"</pre> Script: <i>Terminal</i> <pre>username: abcd Output: username: abcd</pre>
--	---

Read Multiple Inputs using Array:

Script: *hello.sh*

```
#!/bin/bash

echo "Enter names: "
read -a names          # store the input in array
echo "Name: ${names[0]}, ${names[1]}, ${names[2]}"
```

Script: *Terminal*

```
Enter name:
abc def ghi
Output:
Names: abc , def , ghi
```

Create File using Variables:

Script: *hello.sh*

```
#!/bin/sh
echo "What is your name?"
read USER_NAME
echo "Hello $USER_NAME"
echo "I will create you a file called ${USER_NAME}_file"
touch "${USER_NAME}_file"
```

Understanding shell script variables: <https://www.shellscript.sh/variables1.html>

If Statement:

```
# if [ condition ]
# then
#   statement
# fi
```

```
-eq - is equal to - if [ "$a" -eq "$b" ]
-ne - is not equal to - if [ "$a" -ne "$b" ]
-gt - is greater than - if [ "$a" -gt "$b" ]
-ge - is greater than or equal to - if [ "$a" -ge "$b" ]
-lt - is less than - if [ "$a" -lt "$b" ]
-le - is less than or equal to - if [ "$a" -le "$b" ]
< - is less than - (("a" < "b"))
<= - is less than or equal to - (("a" <= "b"))
> - is greater than - (("a" > "b"))
>= - is greater than or equal to - (("a" >= "b"))

string comparison
= - is equal to - if [ "$a" = "$b" ]
== - is equal to - if [ "$a" == "$b" ]
!= - is not equal to - if [ "$a" != "$b" ]
< - is less than, in ASCII alphabetical order - if [[ "$a" < "$b" ]
> - is greater than, in ASCII alphabetical order - if [[ "$a" > "$b" ]
-z - string is null, that is, has zero length
```

Script: <i>hello.sh</i> <pre> #!/bin/bash count=10 if [\$count -eq 9] then echo "condition is true" fi </pre>	Script: <i>hello.sh</i> <pre> #!/bin/bash count=10 if [\$count -gt 9] # if ((\$count > 9)) then echo "condition is true" fi </pre>
Script: <i>hello.sh</i> <pre> #!/bin/bash word=snow if [\$word == "snow"] # if [\$word = "snow"] # if [\$word != "now"] then echo "condition is true" fi </pre>	Script: <i>hello.sh</i> <pre> #!/bin/bash word=snow if [\$word == "snow"] # if [\$word = "snow"] # if [\$word != "now"] then echo "condition is true" fi </pre>
Script: <i>hello.sh</i> <pre> #!/bin/bash word=x if [[\$word < "y"]] then echo "condition is true" fi </pre>	Script: <i>hello.sh</i> <pre> #!/bin/bash word=x if [\$word == "y"] then echo "condition y is true" elif then [\$word == "x"] echo "condition x is true" else echo "condition is false" fi </pre>

Script: *hello.sh*

```
#!/bin/bash
word=x
if [ $word == "y" ]
then
    echo "condition y is true"
elif
then [ $word == "x" ]
    echo "condition x is true"
else
    echo "condition is false"
fi
```

Script: *hello.sh*

```
#!/bin/bash
age = 25
if [ "$age" -gt 18 ] && [ "$age" -lt 30 ]
then
    echo "valid age"
else
    echo " age is not valid"
fi
```

```
# if [ "$age" -gt 18 && "$age" -lt 30 ]
# if [ "$age" -gt 18 ] -a [ "$age" -lt 30 ]
```